

MÓDULO 4: ANÁLISIS EXPLORATORIO Y VISUALIZACIONES

Proyecto: Análisis ETL - Entidad Financiera

Prerequisito: MÓDULO 3 completado (datos limpios en `df_merged`)

OBJETIVO

Responder los **Items 4a-4e** del Trabajo Práctico mediante:

- Análisis estadístico con Pandas
- Visualizaciones con Matplotlib
- Generación de insights de negocio

CONFIGURACIÓN INICIAL DE MATPLOTLIB

► Paso 1: Importar biblioteca y configurar estilo

Nueva celda:

```
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker

# Configuración de estilo
plt.style.use('seaborn-v0_8-darkgrid')
plt.rcParams['figure.figsize'] = (12, 6)
plt.rcParams['font.size'] = 10
plt.rcParams['axes.titlesize'] = 14
plt.rcParams['axes.labelsize'] = 12

print('☑ Matplotlib configurado')
print(f'Estilo: seaborn-v0_8-darkgrid')
print(f'Tamaño por defecto: {plt.rcParams["figure.figsize"]}')

```

¿Qué hace cada configuración?

Configuración	Efecto
<code>plt.style.use()</code>	Aplica estilo visual predefinido
<code>figure.figsize</code>	Tamaño del gráfico en pulgadas (ancho, alto)
<code>font.size</code>	Tamaño de texto general
<code>axes.titlesize</code>	Tamaño del título del gráfico
<code>axes.labelsize</code>	Tamaño de etiquetas de ejes

Salida:

```
☑ Matplotlib configurado
Estilo: seaborn-v0_8-darkgrid
Tamaño por defecto: (12, 6)

```

ITEM 4a: MÉTODO DE PAGO MÁS UTILIZADO

► Paso 1: Análisis estadístico

Nueva celda:

```
print('=' * 80)
print('ITEM 4a: MÉTODO DE PAGO MÁS UTILIZADO')
print('=' * 80)

# Contar frecuencia de cada método
payment_counts = df_merged['payment_method'].value_counts()

print('\n📊 FRECUENCIA DE MÉTODOS DE PAGO:')
print(payment_counts)

print('\n📈 PORCENTAJES:')
payment_pct = (payment_counts / len(df_merged) * 100).round(2)
for method, pct in payment_pct.items():
    print(f'{method:15s}: {pct:6.2f}%')

# Respuesta al ítem
most_used = payment_counts.index[0]
print(f'\n✅ RESPUESTA: El método más utilizado es "{most_used}" con {payment_counts.iloc[0]}')
```

Comandos Pandas usados:

Comando	Función
<code>.value_counts()</code>	Cuenta frecuencia de cada valor único
<code>.index[0]</code>	Primer elemento del índice (categoría)
<code>.iloc[0]</code>	Primer valor numérico

Salida esperada:

```
=====
ITEM 4a: MÉTODO DE PAGO MÁS UTILIZADO
=====
```

```
 FRECUENCIA DE MÉTODOS DE PAGO:
```

```
Cash          44,429
Credit Card   36,366
Debit Card     18,543
Name: payment_method, dtype: int64
```

```
 PORCENTAJES:
```

```
Cash          : 44.72%
Credit Card    : 36.61%
Debit Card     : 18.67%
```

```
☒ RESPUESTA: El método más utilizado es "Cash" con 44,429 transacciones (44.72%)
```

► Paso 2: Visualización con gráfico de barras

Nueva celda:

```
# Crear gráfico de barras
fig, ax = plt.subplots(figsize=(10, 6))

colors = ['#2ecc71', '#3498db', '#e74c3c']
bars = ax.bar(payment_counts.index, payment_counts.values, color=colors, edgecolor='black',

# Personalización
ax.set_title('Distribución de Métodos de Pago', fontsize=16, fontweight='bold', pad=20)
ax.set_xlabel('Método de Pago', fontsize=12, fontweight='bold')
ax.set_ylabel('Número de Transacciones', fontsize=12, fontweight='bold')
ax.yaxis.set_major_formatter(ticker.FuncFormatter(lambda x, p: f'{int(x):,}'))

# Agregar valores sobre las barras
for bar in bars:
    height = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2., height,
            f'{int(height):,}\n({height/len(df_merged)*100:.1f}%)',
            ha='center', va='bottom', fontsize=11, fontweight='bold')

plt.tight_layout()
plt.savefig('../visualizaciones/4a_metodos_pago.png', dpi=300, bbox_inches='tight')
plt.show()

print('☑ Gráfico guardado: visualizaciones/4a_metodos_pago.png')
```

Elementos del gráfico:

Elemento	Código
Crear figura	<code>fig, ax = plt.subplots()</code>
Barras	<code>ax.bar(x, y, color=colors)</code>
Título	<code>ax.set_title('texto')</code>
Etiquetas	<code>ax.set_xlabel()</code> , <code>ax.set_ylabel()</code>
Formato números	<code>ticker.FuncFormatter()</code>
Texto sobre barras	<code>ax.text(x, y, 'texto')</code>
Guardar imagen	<code>plt.savefig('archivo.png', dpi=300)</code>

Colores hexadecimales:

- `#2ecc71` : Verde (Cash)
- `#3498db` : Azul (Credit Card)
- `#e74c3c` : Rojo (Debit Card)



ITEM 4b: GÉNERO CON MÁS COMPRAS

► Paso 1: Análisis estadístico

Nueva celda:

```
print('=' * 80)
print('ITEM 4b: GÉNERO CON MÁS COMPRAS')
print('=' * 80)

# Contar transacciones por género
gender_counts = df_merged['gender'].value_counts()

print('\n📊 TRANSACCIONES POR GÉNERO:')
print(gender_counts)

print('\n📈 PORCENTAJES:')
gender_pct = (gender_counts / len(df_merged) * 100).round(2)
for gender, pct in gender_pct.items():
    print(f'{gender:10s}: {pct:6.2f}%')

# Respuesta
most_purchases = gender_counts.index[0]
print(f'\n✅ RESPUESTA: El género con más compras es "{most_purchases}" con {gender_counts.index[0]}
```

Salida esperada:

```
=====
ITEM 4b: GÉNERO CON MÁS COMPRAS
=====

📊 TRANSACCIONES POR GÉNERO:
Female    59,381
Male      39,957
Name: gender, dtype: int64

📈 PORCENTAJES:
Female    :  59.77%
Male      :  40.23%

✅ RESPUESTA: El género con más compras es "Female" con 59,381 transacciones (59.77%)
```

► Paso 2: Visualización con gráfico de torta

Nueva celda:

```
# Crear gráfico de torta
fig, ax = plt.subplots(figsize=(10, 8))

colors = ['#e91e63', '#2196f3']
explode = (0.05, 0) # Separar primer segmento

wedges, texts, autotexts = ax.pie(
    gender_counts.values,
    labels=gender_counts.index,
    autopct='%1.1f%%',
    startangle=90,
    colors=colors,
    explode=explode,
    shadow=True,
    textprops={'fontsize': 14, 'fontweight': 'bold'}
)

# Personalizar porcentajes
for autotext in autotexts:
    autotext.set_color('white')
    autotext.set_fontsize(16)

ax.set_title('Distribución de Compras por Género', fontsize=16, fontweight='bold', pad=20)

# Agregar leyenda con conteos
legend_labels = [f'{gender}: {count:,} transacciones'
                  for gender, count in zip(gender_counts.index, gender_counts.values)]
ax.legend(legend_labels, loc='upper right', fontsize=12)

plt.tight_layout()
plt.savefig('../visualizaciones/4b_genero_compras.png', dpi=300, bbox_inches='tight')
plt.show()

print('☑ Gráfico guardado: visualizaciones/4b_genero_compras.png')
```

Elementos del gráfico de torta:

Parámetro	Descripción
<code>labels</code>	Etiquetas de cada segmento
<code>autopct</code>	Formato de porcentajes (%.1f%%)
<code>startangle</code>	Ángulo inicial (90 = vertical)
<code>explode</code>	Separar segmentos (0.05 = 5%)
<code>shadow=True</code>	Agregar sombra

ITEM 4c: CATEGORÍA DE PRODUCTO CON MAYOR FACTURACIÓN

► Paso 1: Cálculo de facturación por categoría

Nueva celda:

```
print('=' * 80)
print('ITEM 4c: CATEGORÍA CON MAYOR FACTURACIÓN')
print('=' * 80)

# Sumar ventas por categoría
revenue_by_category = df_merged.groupby('product_category')['total_sale'].sum().sort_values(ascending=True)

print('\n💰 FACTURACIÓN POR CATEGORÍA:')
for category, revenue in revenue_by_category.items():
    print(f'{category:20s}: ${revenue:>15,.2f}')

print('\n📊 TOTAL GENERAL:')
print(f'${revenue_by_category.sum():,.2f}')

# Respuesta
top_category = revenue_by_category.index[0]
top_revenue = revenue_by_category.iloc[0]
print(f'\n✅ RESPUESTA: "{top_category}" tiene la mayor facturación: ${top_revenue:,.2f} (top 1)')
```

Comando clave: `.groupby()`

```
df.groupby('columna')['otra_columna'].sum()
```

- Agrupa por valores únicos de `columna`
- Suma valores de `otra_columna` para cada grupo
- Retorna Series con resultados

Salida esperada:

```
=====
ITEM 4c: CATEGORÍA CON MAYOR FACTURACIÓN
=====
```

💰 FACTURACIÓN POR CATEGORÍA:

```
Clothing      : $113,802,356.68
Technology    : $ 75,137,542.03
Shoes         : $ 50,036,791.35
Books         : $ 49,866,729.73
Cosmetics     : $ 24,931,486.48
Toys          : $ 24,932,282.18
```

 TOTAL GENERAL:

```
$338,707,188.45
```

☒ RESPUESTA: "Clothing" tiene la mayor facturación: \$113,802,356.68 (33.60% del total)

► Paso 2: Visualización con gráfico de barras horizontal

Nueva celda:

```
# Crear gráfico de barras horizontal
fig, ax = plt.subplots(figsize=(12, 8))

colors_gradient = plt.cm.viridis(np.linspace(0.3, 0.9, len(revenue_by_category)))
bars = ax.barh(revenue_by_category.index, revenue_by_category.values, color=colors_gradient,

# Personalización
ax.set_title('Facturación por Categoría de Producto', fontsize=16, fontweight='bold', pad=20)
ax.set_xlabel('Facturación Total (USD)', fontsize=12, fontweight='bold')
ax.set_ylabel('Categoría', fontsize=12, fontweight='bold')
ax.xaxis.set_major_formatter(ticker.FuncFormatter(lambda x, p: f'${x/1e6:.0f}M'))

# Agregar valores al final de cada barra
for i, (category, revenue) in enumerate(revenue_by_category.items()):
    ax.text(revenue, i, f' ${revenue/1e6:.1f}M',
            va='center', fontsize=11, fontweight='bold')

# Invertir eje Y para mostrar mayor arriba
ax.invert_yaxis()

plt.tight_layout()
plt.savefig('../visualizaciones/4c_facturacion_categoria.png', dpi=300, bbox_inches='tight')
plt.show()

print('✅ Gráfico guardado: visualizaciones/4c_facturacion_categoria.png')
```

Elementos nuevos:

Código	Función
<code>ax.barh()</code>	Barras horizontales
<code>plt.cm.viridis()</code>	Paleta de colores
<code>np.linspace(0.3, 0.9, n)</code>	Generar <code>n</code> valores entre 0.3 y 0.9
<code>\${x/1e6:.0f}M</code>	Mostrar millones (1e6 = 1,000,000)
<code>ax.invert_yaxis()</code>	Invertir eje vertical



ITEM 4d: TICKET PROMEDIO POR UBICACIÓN

► Paso 1: Calcular promedios

Nueva celda:

```
print('=' * 80)
print('ITEM 4d: TICKET PROMEDIO POR UBICACIÓN')
print('=' * 80)

# Calcular promedio de venta por ubicación
avg_ticket_location = df_merged.groupby('location')['total_sale'].mean().sort_values(ascending=True)

print('\n📊 TICKET PROMEDIO POR UBICACIÓN:')
for location, avg in avg_ticket_location.items():
    # Contar transacciones
    transactions = len(df_merged[df_merged['location'] == location])
    print(f'{location:15s}: ${avg:>8,.2f} ({transactions:,} transacciones)')

# Respuesta
highest_avg_location = avg_ticket_location.index[0]
highest_avg = avg_ticket_location.iloc[0]
print(f'\n✅ RESPUESTA: "{highest_avg_location}" tiene el ticket promedio más alto: ${highest_avg:>8,.2f}')
```

Salida esperada:

```
=====
ITEM 4d: TICKET PROMEDIO POR UBICACIÓN
=====

📊 TICKET PROMEDIO POR UBICACIÓN:
Ankara      : $3,459.23 (8,283 transacciones)
Izmir       : $3,443.45 (16,616 transacciones)
Istanbul    : $3,385.23 (74,439 transacciones)

✅ RESPUESTA: "Ankara" tiene el ticket promedio más alto: $3,459.23
```

► Paso 2: Visualización con boxplot

Nueva celda:

```
# Preparar datos para boxplot
locations = df_merged['location'].unique()
data_by_location = [df_merged[df_merged['location'] == loc]['total_sale'].values
                     for loc in locations]

# Crear boxplot
fig, ax = plt.subplots(figsize=(12, 8))

bp = ax.boxplot(data_by_location, labels=locations, patch_artist=True,
                notch=True, showmeans=True,
                boxprops=dict(facecolor='lightblue', edgecolor='black', linewidth=1.5),
                whiskerprops=dict(color='black', linewidth=1.5),
                capprops=dict(color='black', linewidth=1.5),
                medianprops=dict(color='red', linewidth=2),
                meanprops=dict(marker='D', markerfacecolor='green', markeredgecolor='black'))

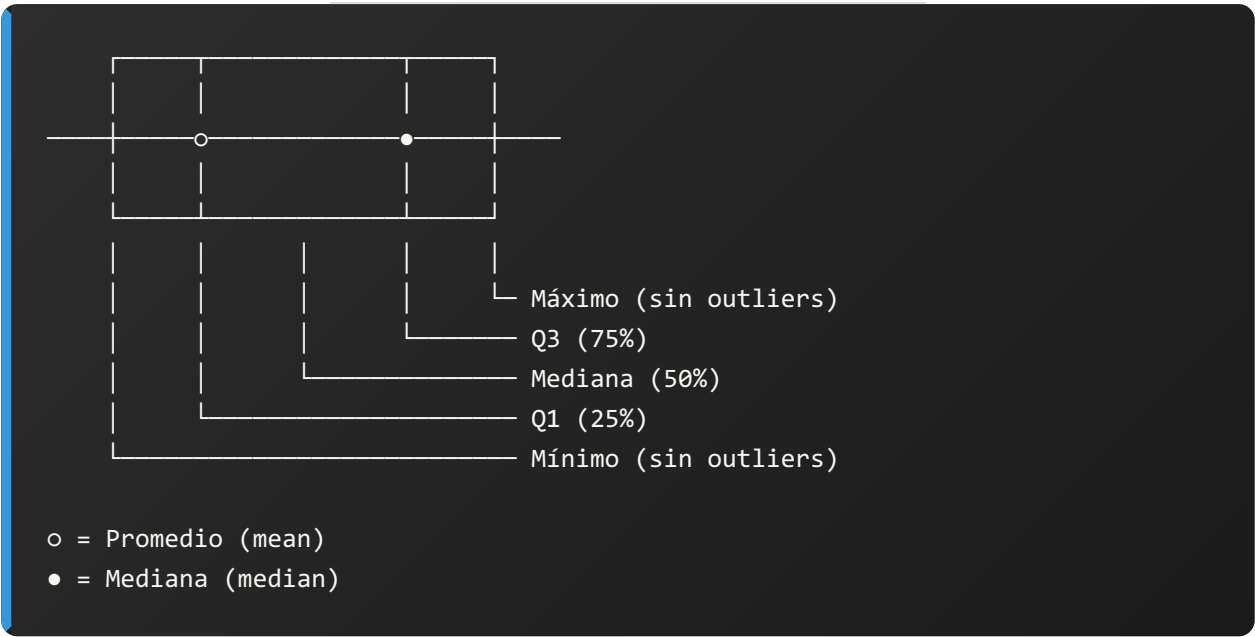
# Personalización
ax.set_title('Distribución del Ticket de Venta por Ubicación', fontsize=16, fontweight='bold')
ax.set_xlabel('Ubicación', fontsize=12, fontweight='bold')
ax.set_ylabel('Monto de Venta (USD)', fontsize=12, fontweight='bold')
ax.yaxis.set_major_formatter(ticker.FuncFormatter(lambda x, p: f'${x:,.0f}'))
ax.grid(axis='y', alpha=0.3)

# Leyenda
ax.legend([bp['medians'][0], bp['means'][0]],
          ['Mediana', 'Promedio'],
          loc='upper right', fontsize=10)

plt.tight_layout()
plt.savefig('../visualizaciones/4d_ticket_promedio_ubicacion.png', dpi=300, bbox_inches='tight')
plt.show()

print('☑ Gráfico guardado: visualizaciones/4d_ticket_promedio_ubicacion.png')
```

¿Qué es un boxplot?



Elementos del boxplot:

Parámetro	Descripción
patch_artist=True	Cajas con color
notch=True	Muesca en la mediana
showmeans=True	Mostrar promedio
boxprops	Estilo de la caja
medianprops	Estilo de la línea de mediana
meanprops	Estilo del marcador de promedio



ITEM 4e: CATEGORÍA CON MAYOR PRECIO PROMEDIO

► Paso 1: Calcular precios promedio

Nueva celda:

```
print('=' * 80)
print('ITEM 4e: CATEGORÍA CON MAYOR PRECIO PROMEDIO')
print('=' * 80)

# Calcular precio promedio por categoría
avg_price_category = df_merged.groupby('product_category')['price'].mean().sort_values(ascendi

print('\n💰 PRECIO PROMEDIO POR CATEGORÍA:')
for category, avg_price in avg_price_category.items():
    # Estadísticas adicionales
    cat_data = df_merged[df_merged['product_category'] == category]['price']
    print(f'{category:20s}: ${avg_price:>8,.2f} (min: ${cat_data.min():.2f}, max: ${cat_data

# Respuesta
highest_price_category = avg_price_category.index[0]
highest_price = avg_price_category.iloc[0]
print(f'\n✅ RESPUESTA: "{highest_price_category}" tiene el precio promedio más alto: ${high
```

Salida esperada:


```
=====
ITEM 4e: CATEGORÍA CON MAYOR PRECIO PROMEDIO
=====
```

💰 PRECIO PROMEDIO POR CATEGORÍA:

Technology	: \$3,157.35	(min: \$25.00, max: \$9,999.00)
Shoes	: \$2,522.44	(min: \$25.00, max: \$5,000.00)
Clothing	: \$2,508.16	(min: \$25.00, max: \$5,000.00)
Books	: \$2,499.00	(min: \$25.00, max: \$5,000.00)
Cosmetics	: \$1,250.16	(min: \$25.00, max: \$2,500.00)
Toys	: \$1,249.92	(min: \$25.00, max: \$2,500.00)

☑ RESPUESTA: "Technology" tiene el precio promedio más alto: \$3,157.35

► Paso 2: Visualización con gráfico de barras con error bars

Nueva celda:

```
# Calcular desviación estándar para error bars
std_price_category = df_merged.groupby('product_category')['price'].std()

# Crear gráfico
fig, ax = plt.subplots(figsize=(12, 8))

x_pos = np.arange(len(avg_price_category))
colors = plt.cm.Spectral(np.linspace(0.2, 0.8, len(avg_price_category)))

bars = ax.bar(x_pos, avg_price_category.values,
              yerr=std_price_category[avg_price_category.index].values,
              color=colors, edgecolor='black', linewidth=1.5,
              capsize=10, error_kw={'linewidth': 2, 'ecolor': 'black'})

# Personalización
ax.set_title('Precio Promedio por Categoría de Producto', fontsize=16, fontweight='bold', pad=10)
ax.set_xlabel('Categoría', fontsize=12, fontweight='bold')
ax.set_ylabel('Precio Promedio (USD)', fontsize=12, fontweight='bold')
ax.set_xticks(x_pos)
ax.set_xticklabels(avg_price_category.index, rotation=45, ha='right')
ax.yaxis.set_major_formatter(ticker.FuncFormatter(lambda x, p: f'${x:,.0f}'))

# Agregar valores sobre las barras
for i, (bar, value) in enumerate(zip(bars, avg_price_category.values)):
    ax.text(bar.get_x() + bar.get_width()/2., value + std_price_category.iloc[i],
            f'${value:,.0f}', ha='center', va='bottom', fontsize=11, fontweight='bold')

plt.tight_layout()
plt.savefig('../visualizaciones/4e_precio_promedio_categoria.png', dpi=300, bbox_inches='tight')
plt.show()

print('☑ Gráfico guardado: visualizaciones/4e_precio_promedio_categoria.png')
```

Error bars:

- **yerr** : Valores de error (desviación estándar)
- **capsize** : Tamaño de las líneas horizontales
- **error_kw** : Estilo de las barras de error



ANÁLISIS ADICIONALES RELEVANTES

► 1. Evolución temporal de ventas

Nueva celda:

```
# Ventas por mes
monthly_sales = df_merged.groupby(df_merged['invoice_date'].dt.to_period('M'))['total_sale'].sum()

fig, ax = plt.subplots(figsize=(14, 6))

ax.plot(range(len(monthly_sales)), monthly_sales.values,
        marker='o', linewidth=2.5, markersize=8, color='#2c3e50')

ax.set_title('Evolución de Ventas Mensuales', fontsize=16, fontweight='bold', pad=10)
ax.set_xlabel('Mes', fontsize=12, fontweight='bold')
ax.set_ylabel('Ventas Totales (USD)', fontsize=12, fontweight='bold')
ax.yaxis.set_major_formatter(ticker.FuncFormatter(lambda x, p: f'${x/1e6:.1f}M'))
ax.grid(alpha=0.3)

# Etiquetas de meses (cada 3 meses)
months_labels = [str(p) for p in monthly_sales.index]
ax.set_xticks(range(0, len(monthly_sales), 3))
ax.set_xticklabels(months_labels[::3], rotation=45, ha='right')

plt.tight_layout()
plt.savefig('../visualizaciones/ventas_mensuales.png', dpi=300, bbox_inches='tight')
plt.show()

print('☑ Gráfico guardado: visualizaciones/ventas_mensuales.png')
```

► 2. Top 10 clientes por facturación

Nueva celda:

```
# Clientes con mayor gasto
top_customers = df_merged.groupby('customer_id')['total_sale'].sum().sort_values(ascending=False)

fig, ax = plt.subplots(figsize=(12, 8))

bars = ax.barh(range(len(top_customers)), top_customers.values, color='#16a085', edgecolor='black')

ax.set_title('Top 10 Clientes por Facturación Total', fontsize=16, fontweight='bold', pad=20)
ax.set_xlabel('Facturación Total (USD)', fontsize=12, fontweight='bold')
ax.set_ylabel('Cliente ID', fontsize=12, fontweight='bold')
ax.set_yticks(range(len(top_customers)))
ax.set_yticklabels([f'ID {customer_id}' for customer_id in top_customers.index])
ax.xaxis.set_major_formatter(ticker.FuncFormatter(lambda x, p: f'${x/1e3:.0f}K'))

# Agregar valores
for i, value in enumerate(top_customers.values):
    ax.text(value, i, f' ${value/1e3:.1f}K', va='center', fontsize=10, fontweight='bold')

ax.invert_yaxis()

plt.tight_layout()
plt.savefig('../visualizaciones/top_clientes.png', dpi=300, bbox_inches='tight')
plt.show()

print('☑ Gráfico guardado: visualizaciones/top_clientes.png')
```

► 3. Distribución de edad de clientes

Nueva celda:

```
# Histograma de edad
fig, ax = plt.subplots(figsize=(12, 6))

n, bins, patches = ax.hist(df_merged['age'], bins=30, color='#3498db', edgecolor='black', linewidth=1)

# Colorear barras por gradiente
cm = plt.cm.viridis
for i, patch in enumerate(patches):
    patch.set_facecolor(cm(i / len(patches)))

ax.set_title('Distribución de Edad de Clientes', fontsize=16, fontweight='bold', pad=20)
ax.set_xlabel('Edad', fontsize=12, fontweight='bold')
ax.set_ylabel('Frecuencia', fontsize=12, fontweight='bold')
ax.yaxis.set_major_formatter(ticker.FuncFormatter(lambda x, p: f'{int(x):,}'))

# Línea de promedio
mean_age = df_merged['age'].mean()
ax.axvline(mean_age, color='red', linestyle='--', linewidth=2, label=f'Promedio: {mean_age:.1f}')
ax.legend(fontsize=12)

plt.tight_layout()
plt.savefig('../visualizaciones/distribucion_edad.png', dpi=300, bbox_inches='tight')
plt.show()

print('☑ Gráfico guardado: visualizaciones/distribucion_edad.png')
print(f'Edad promedio: {mean_age:.2f} años')
print(f'Edad mínima: {df_merged["age"].min():.0f} años')
print(f'Edad máxima: {df_merged["age"].max():.0f} años')
```



RESUMEN DE VISUALIZACIONES CREADAS

#	Archivo	Tipo	Item
1	4a_metodos_pago.png	Barras	4a
2	4b_genero_compras.png	Torta	4b
3	4c_facturacion_categoria.png	Barras horizontales	4c
4	4d_ticket_promedio_ubicacion.png	Boxplot	4d
5	4e_precio_promedio_categoria.png	Barras con error	4e
6	ventas_mensuales.png	Línea	Temporal
7	top_clientes.png	Barras horizontales	Negocio
8	distribucion_edad.png	Histograma	Demográfico



COMMIT A GIT

```
# Guardar visualizaciones
git add visualizaciones/*.png
git add notebooks/analisis_etl.ipynb
git commit -m "Agregar análisis Items 4a-4e y 8 visualizaciones"
git push origin main
```



COMANDOS MATPLOTLIB APRENDIDOS

Comando	Función
<code>plt.subplots(figsize=(w,h))</code>	Crear figura y ejes
<code>ax.bar(x, y)</code>	Gráfico de barras vertical
<code>ax.barh(x, y)</code>	Gráfico de barras horizontal
<code>ax.pie(values, labels)</code>	Gráfico de torta
<code>ax.boxplot(data)</code>	Gráfico de caja
<code>ax.plot(x, y)</code>	Gráfico de línea
<code>ax.hist(data, bins)</code>	Histograma
<code>ax.set_title('texto')</code>	Título del gráfico
<code>ax.set_xlabel('texto')</code>	Etiqueta eje X
<code>ax.set_ylabel('texto')</code>	Etiqueta eje Y
<code>plt.savefig('archivo.png', dpi=300)</code>	Guardar imagen
<code>ticker.FuncFormatter(lambda)</code>	Formato personalizado de ejes



PRÓXIMOS PASOS

Ver **MÓDULO 5: IMPLEMENTACIÓN SQL** para:

- Diseño de esquema de base de datos
- Creación de tablas en SQLite
- Consultas SQL de análisis
- Integración con Python

Documento creado: 8 de noviembre de 2025

Parte de: Documentación Técnica Completa - Proyecto ETL TSCDIA